

# **Estrutura de um Sistema Operacional**

Internamente, os sistemas operacionais podem ter diferentes estruturas, como monolítica, em camadas, microkernel, cliente-servidor e exokernel. Dois desses modelos são apresentados a seguir.

## **Kernel Monolítico**

De longe a organização mais comum, na abordagem monolítica todo o sistema operacional é executado como um único programa em modo kernel. O sistema operacional é escrito como uma coleção de procedimentos, ligados entre si em um único grande programa executável binário. Quando essa técnica é utilizada, cada procedimento do sistema pode chamar qualquer outro, desde que este forneça alguma computação útil que o primeiro necessite.

A possibilidade de chamar qualquer procedimento é muito eficiente, mas ter milhares de procedimentos que podem se chamar mutuamente sem restrições pode resultar em um sistema difícil de compreender e de manter. Além disso, uma falha em qualquer um desses procedimentos pode derrubar todo o sistema operacional.

Para construir o programa objeto do sistema operacional com essa abordagem, primeiro compila-se cada procedimento individual (ou os arquivos que os contêm) e, em seguida, todos são ligados em um único arquivo executável por meio do ligador do sistema. Em termos de ocultação de informação, praticamente não há nenhuma — todo procedimento é visível para os demais (em contraste com estruturas modulares, nas quais grande parte das informações fica encapsulada dentro de módulos, e apenas pontos de entrada definidos podem ser acessados externamente).

Mesmo em sistemas monolíticos, contudo, é possível introduzir alguma estrutura. Os serviços (chamadas de sistema) fornecidos pelo sistema operacional são requisitados colocando-se os parâmetros em um local bem definido (por exemplo, na pilha) e executando uma instrução de trap. Essa instrução alterna a máquina do modo usuário para o modo kernel e transfere o controle ao sistema operacional.

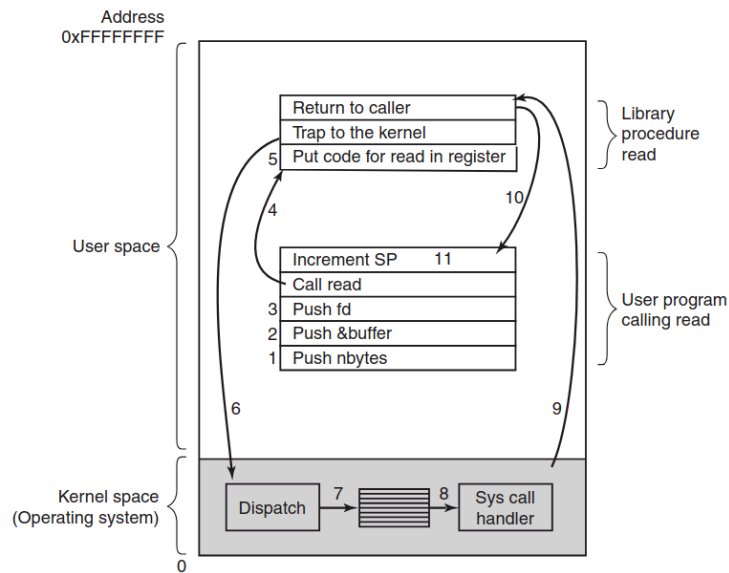


Figure 1-17. The 11 steps in making the system call read(fd, buffer, nbytes).

Essa organização sugere uma estrutura básica para o sistema operacional:

1. Um programa principal que invoca o procedimento de serviço solicitado.
2. Um conjunto de procedimentos de serviço que implementam as chamadas de sistema.
3. Um conjunto de procedimentos utilitários que auxiliam os procedimentos de serviço.

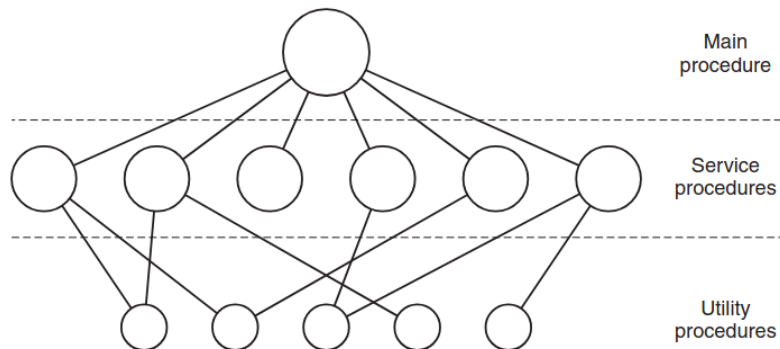


Figure 1-24. A simple structuring model for a monolithic system.

Nesse modelo, para cada chamada de sistema existe um procedimento de serviço responsável por tratá-la e executá-la. Os procedimentos utilitários realizam tarefas necessárias a vários procedimentos de serviço, como a obtenção de dados de programas do usuário.

Além do núcleo do sistema operacional carregado durante a inicialização do computador, muitos sistemas operacionais oferecem suporte a extensões carregáveis, como drivers de

dispositivos de E/S e sistemas de arquivos. Esses componentes são carregados sob demanda. Em sistemas UNIX, são chamados de bibliotecas compartilhadas; no Windows, são conhecidos como DLLs (Dynamic-Link Libraries).

## Microkernels

Na abordagem em camadas, os projetistas podem decidir onde traçar a fronteira entre kernel e usuário. Tradicionalmente, todas as camadas eram incluídas no kernel, mas isso não é obrigatório. Na verdade, há fortes argumentos para manter o mínimo possível em modo kernel, pois erros no kernel podem derrubar o sistema instantaneamente. Em contraste, processos em modo usuário podem ser configurados com menos privilégios, de modo que falhas neles não sejam fatais.

A ideia central do projeto de microkernel é alcançar alta confiabilidade dividindo o sistema operacional em pequenos módulos bem definidos, dos quais apenas um, o microkernel, executa em modo kernel, enquanto os demais operam como processos comuns em modo usuário, com privilégios reduzidos. Em particular, ao executar cada driver de dispositivo e sistema de arquivos como um processo separado em modo usuário, um erro em um desses componentes pode derrubá-lo individualmente, mas não compromete todo o sistema.

Assim, um erro em um driver de áudio pode fazer com que o som fique distorcido ou pare de funcionar, mas não causará a falha do computador. Em contraste, em um sistema monolítico, no qual todos os drivers estão no kernel, um driver de áudio defeituoso pode facilmente acessar um endereço de memória inválido e travar completamente o sistema.

Microkernels são predominantes em aplicações de tempo real, industriais, aeronáuticas e militares, que são críticas e exigem altíssima confiabilidade. Alguns dos microkernels mais conhecidos incluem Integrity, K42, L4, PikeOS, QNX, Symbian e MINIX 3. A seguir, apresenta-se uma breve visão geral do MINIX 3, que leva a ideia de modularidade ao extremo, dividindo a maior parte do sistema operacional em diversos processos independentes em modo usuário. O MINIX 3 é um sistema compatível com POSIX, de código aberto e disponível gratuitamente.

O microkernel do MINIX 3 possui apenas cerca de 12.000 linhas de código em C e aproximadamente 1.400 linhas em assembly para funções de baixo nível, como tratamento de interrupções e troca de processos. O código em C gerencia e escalona processos, trata a comunicação entre processos (por meio da troca de mensagens) e oferece cerca de 40 chamadas de kernel para permitir que o restante do sistema operacional funcione. Essas chamadas realizam funções como associar tratadores a interrupções, mover dados entre espaços de endereçamento e instalar mapas de memória para novos processos.

## Linux vs Minix

O Linux é um sistema operacional de código aberto baseado em um kernel monolítico modular, amplamente utilizado em servidores, dispositivos móveis e sistemas embarcados. Seu desenvolvimento foi iniciado em 1991 por Linus Torvalds, então estudante de Ciência da Computação, com o objetivo de criar um sistema funcional inspirado no UNIX, mas acessível e colaborativo. Ao longo dos anos, o Linux evoluiu por meio da contribuição de uma vasta comunidade global de desenvolvedores, tornando-se um dos pilares da infraestrutura digital contemporânea. Sua arquitetura prioriza desempenho e flexibilidade, permitindo a incorporação de módulos dinamicamente carregáveis e ampla portabilidade entre diferentes arquiteturas de hardware.

O MINIX, por sua vez, é um sistema operacional de propósito educacional desenvolvido por Andrew S. Tanenbaum no final da década de 1980. Projetado com base na arquitetura de microkernel, o MINIX tem como principal objetivo facilitar o ensino e a compreensão dos conceitos fundamentais de sistemas operacionais. Sua estrutura modular enfatiza a separação entre componentes do sistema, promovendo maior confiabilidade e facilidade de análise. Embora inicialmente concebido para fins acadêmicos, versões mais recentes, como o MINIX 3, passaram a focar também em robustez e tolerância a falhas, sendo utilizado como base para pesquisas em sistemas confiáveis.

Uma famosa discussão entre Andrew S. Tanenbaum e Linus Torvalds ocorreu em 1992, na Usenet (grupo *comp.os.minix*), e tornou-se um marco no debate sobre **arquitetura de sistemas operacionais**, especialmente entre **microkernels** e **kernels monolíticos**.

A discussão surgiu quando Tanenbaum criticou publicamente o design do Linux.

## Posição de Tanenbaum

Tanenbaum argumentava que:

- O modelo **monolítico era obsoleto** (*"Linux is obsolete."*, ele dizia)
- O futuro estava nos **microkernels**, por:
  - maior modularidade
  - melhor confiabilidade
  - isolamento de falhas
- O Linux era:
  - pouco portátil (dependente de x86 - *"It is tied to the 386."*)
  - um projeto mais "hobby" do que acadêmico

Em síntese, microkernels seriam tecnicamente superiores e mais adequados a longo prazo.

## Posição de Linus

Torvalds respondeu defendendo o Linux:

- O modelo monolítico era:
  - **mais simples**
  - **mais eficiente** (melhor desempenho - *"Performance matters."*)
- Microkernels tinham:
  - overhead de comunicação (IPC)
  - pior desempenho prático na época
- Linux não buscava ser "teoricamente ideal" (*"I'm doing a free OS (just a hobby...)"*), mas:
  - **funcional**
  - **rápido**
  - **útil no mundo real**

Em síntese, pragmatismo e desempenho superaram elegância teórica.

## Pontos centrais do debate

| Tema           | Tanenbaum (MINIX)   | Torvalds (Linux)   |
|----------------|---------------------|--------------------|
| Arquitetura    | Microkernel         | Monolítico         |
| Foco           | Elegância, pesquisa | Praticidade        |
| Confiabilidade | Alta (isolamento)   | Menor              |
| Desempenho     | Inferior (na época) | Superior           |
| Portabilidade  | Alta                | Inicialmente baixa |

## Impacto histórico

- O debate tornou-se **icônico na área de Sistemas Operacionais**
- Influenciou gerações de pesquisadores e engenheiros
- Mostrou o conflito entre:
  - **engenharia prática vs projeto teórico**

## O que aconteceu depois?

- O Linux se tornou dominante (servidores, mobile, cloud)
- Microkernels continuaram relevantes em:
  - sistemas críticos (ex: QNX)
- Surgiram **kernels híbridos** (ex: macOS)

Curiosamente, muitas ideias de microkernel foram incorporadas indiretamente em sistemas modernos.

## Conclusão

O debate não teve um “vencedor absoluto”, mas evidenciou que **trade-offs entre desempenho, modularidade e confiabilidade** são centrais no projeto de sistemas operacionais.

- Tanenbaum estava certo em termos **conceituais**
- Torvalds venceu em termos **práticos e de adoção**

Fontes:

[TANENBAUM, Andrew S.; BOS, Herbert. \*Modern Operating Systems\*. 4. ed. Boston: Pearson. 2015.](#)

[Open Sources: Voices from the Open Source Revolution](#)

[https://en.wikipedia.org/wiki/Tanenbaum%E2%80%93Torvalds\\_debate](https://en.wikipedia.org/wiki/Tanenbaum%E2%80%93Torvalds_debate)